

CSC215 Lab11 – Browser Exploits

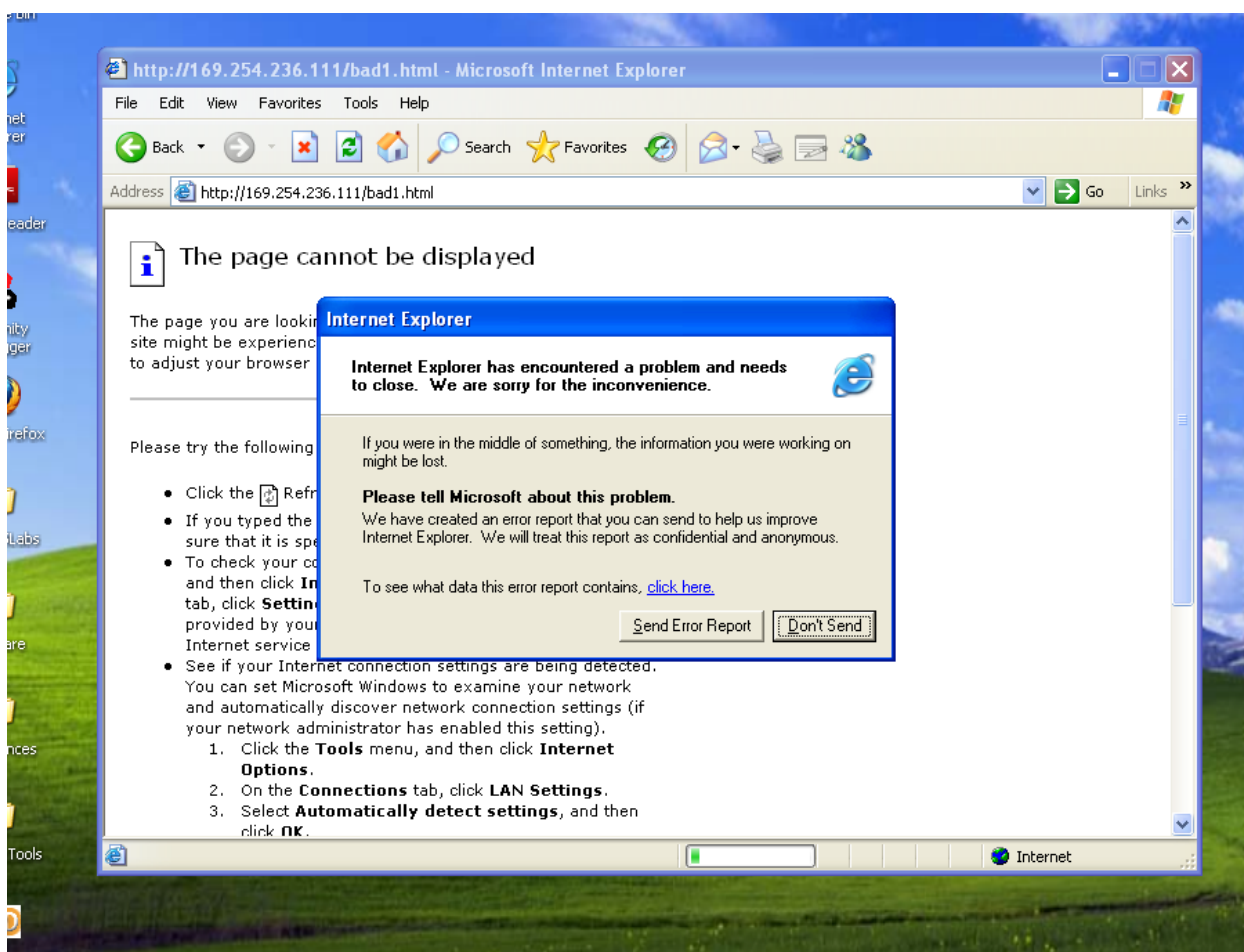
Total: 150pt

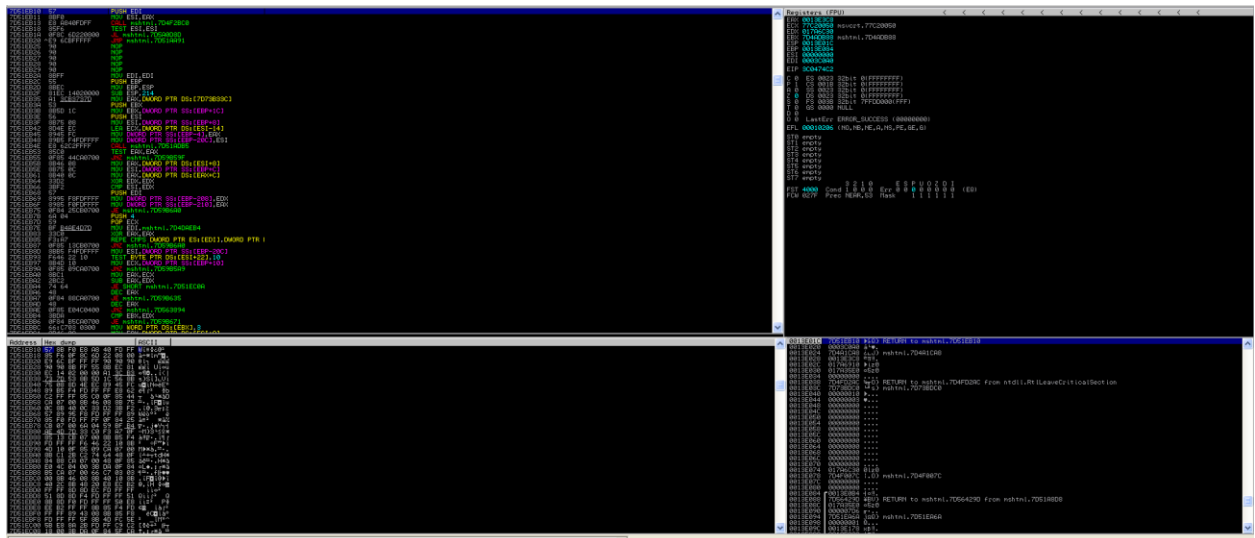
What to turn in:

- (1) CSC215Lab17.docx
- (2) bad3.html

Note: the files needed for this lab are located in Backtrack5 ~/Desktop/Browser folder.

1. [10pt] Repeat the exploit of bad1.html on WinXP IE 6.0. Note bad1.html is located in BT5 . Paste the success screen below.





2. [10pt] Repeat the exploit of [bad2.html](#) on IE. You need to show that the debugger **stops at the first instruction of shell code**. Paste the success screenshot below. (The expected shell code is **NOP NOP NOP** If you press F9, you should see **INT3 INT3 INT3 ...**)

```

3C060025 CC      INT3
3C060026 CC      INT3
3C060027 CC      INT3
3C060028 CC      INT3
3C060029 CC      INT3
3C06002A CC      INT3
3C06002B CC      INT3
3C06002C CC      INT3
3C06002D CC      INT3
3C06002E CC      INT3
3C06002F CC      INT3
3C060030 CC      INT3
3C060031 CC      INT3
3C060032 CC      INT3
3C060033 CC      INT3
3C060034 CC      INT3
3C060035 CC      INT3
3C060036 CC      INT3
3C060037 CC      INT3
3C060038 CC      INT3
3C060039 CC      INT3
3C06003A CC      INT3
3C06003B CC      INT3
3C06003C CC      INT3
3C06003D CC      INT3
3C06003E CC      INT3
3C06003F CC      INT3
3C060040 CC      INT3
3C060041 CC      INT3
3C060042 CC      INT3
3C060043 CC      INT3
3C060044 CC      INT3
3C060045 CC      INT3
3C060046 CC      INT3
3C060047 CC      INT3
3C060048 CC      INT3
3C060049 CC      INT3
3C06004A CC      INT3
3C06004B CC      INT3
3C06004C CC      INT3
3C06004D CC      INT3
3C06004E CC      INT3
3C06004F CC      INT3
3C060050 CC      INT3
3C060051 CC      INT3
3C060052 CC      INT3
3C060053 CC      INT3
3C060054 CC      INT3
3C060055 CC      INT3
3C060056 CC      INT3
3C060057 CC      INT3
3C060058 CC      INT3
3C060059 CC      INT3
3C06005A CC      INT3
3C06005B CC      INT3

```

3. [20pt] Install **mangleme** (source code in ~/Desktop/Browser of BT5) as a **CGI-BIN** service. Paste the success screenshot below. (hint: you need to find out where cgi-bin files are installed by reading the configuration file of Apache in /etc).

```
root@bt:~/Desktop/Browser/mangleme-1.2# sudo find / -type d -name "cgi-bin"
/usr/lib/cgi-bin
/usr/lib/w3m/cgi-bin
/pentest/web/xssfuzz/cgi-bin
root@bt:~/Desktop/Browser/mangleme-1.2# ls
gallery Makefile mangle.cgi mangle.cgi.c README remangle.cgi.c tags.h
root@bt:~/Desktop/Browser/mangleme-1.2# cp mangle.cgi /usr/lib/cgi-bin
root@bt:~/Desktop/Browser/mangleme-1.2# cd /usr/lib/cgi-bin
bash: cd: /usr/lib/cgi-bin: Not a directory
root@bt:~/Desktop/Browser/mangleme-1.2# ls /usr/lib/cgi-bin
/usr/lib/cgi-bin
root@bt:~/Desktop/Browser/mangleme-1.2# ls -l /usr/lib/cgi-bin
-rwxr-xr-x 1 root root 18299 2025-12-04 19:24 /usr/lib/cgi-bin
root@bt:~/Desktop/Browser/mangleme-1.2# ls
gallery Makefile mangle.cgi mangle.cgi.c README remangle.cgi.c tags.h
root@bt:~/Desktop/Browser/mangleme-1.2# cp mangle.cgi /usr/lib/c
caca/          cgi-bin/      crda/          crtn.o
casper/        command-not-found crt1.o         cruft/
cg-bin         compat-ld/    crt1.o         cups/
root@bt:~/Desktop/Browser/mangleme-1.2# cp mangle.cgi /usr/lib/c
caca/          cgi-bin/      crda/          crtn.o
casper/        command-not-found crt1.o         cruft/
cg-bin         compat-ld/    crt1.o         cups/
root@bt:~/Desktop/Browser/mangleme-1.2# cp mangle.cgi /usr/lib/cgi-bin/
root@bt:~/Desktop/Browser/mangleme-1.2# ls -l /usr/lib/cgi-bin
total 20
-rwxr-xr-x 1 root root 18299 2025-12-04 19:25 mangle.cgi
root@bt:~/Desktop/Browser/mangleme-1.2#
```

4. [20pt] Propose an enhancement on the browser's **JavaScript engine** so that it would be able to detect heap spray attacks. Your proposal should contain enough technical details to help reviewers to verify its feasibility.

A feasible enhancement for detecting heap-spray attacks in a browser's JavaScript engine is to add a **Heap-Spray Anomaly Detection (HSAD)** layer integrated into the existing allocator and garbage collector. The engine would record low-overhead telemetry about recent allocations—such as bucketized size distributions, allocation rates, and repetition patterns per script origin—within a rolling time window. Using this metadata, the engine applies heuristics that flag suspicious behavior, including unusually large numbers of identical-sized buffers (e.g., large strings or TypedArrays), low-entropy or highly repetitive byte patterns detected through lightweight randomized sampling, and abnormal heap growth where objects are rapidly allocated but seldom referenced. When these signals exceed a defined threshold, the engine can disrupt the attack by throttling large allocations from the origin, triggering earlier incremental GC cycles to break attacker-controlled layout, or reporting an event to the browser's threat subsystem, with script termination as a last resort. This approach is practical because it builds on existing memory-management components, uses probabilistic sampling to keep overhead low, and avoids interfering with legitimate workloads, which rarely generate massive volumes of uniform, low-entropy allocations in short bursts.

- 5*. [90pt] Now modify **bad2.html** (save it as **bad3.html**) so that IE would pop an instance of calculator. (Hint: use **msfpayload** and **msfencode** in metasploit. Pay attention to the Unicode encoding of javascript and little endianness. Also for avoid bad chars, only avoid `"\x00"`, don't avoid `"\x90"`). If your calc.exe does not pop but you can trace into the shell code and prove your case is

